



A. MỤC TIÊU:

- Giới thiệu về nguồn tài nguyên Google Maps
- Ứng dụng truy cập vào bản đồ, định vị, tìm đường đi và đánh dấu vị trí

B. DỤNG CỤ - THIẾT BỊ THỰC HÀNH CHO MỘT SV:

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

1.1. Giới thiệu về Google Maps

- Google Maps là một dịch vụ bản đồ trực tuyến do Google cung cấp, ra mắt lần đầu vào năm 2005. Nó cho phép người dùng xem bản đồ, định vị, tìm đường đi, đánh dấu định vị và khám phá thông tin địa lý trên toàn thế giới.
- Các tính năng chính:
 - Hiển thị bản đồ: bản đồ 2D, 3D, chế độ vệ tinh (Satellite), và Street View.
 - Định vị: xác định vị trí hiện tại bằng GPS.
 - Tìm đường đi (Directions): cung cấp tuyến đường tối ưu cho xe hơi, đi bộ, xe đạp, hoặc phương tiện công cộng.
 - Đánh dấu địa điểm: Hiển thị các điểm quan tâm (POI) như nhà hàng, khách sạn, trạm xăng,...
 - Tích hợp API: Cho phép lập trình viên nhúng bản đồ vào ứng dụng hoặc website.
- Trong ứng dụng di động, Google Maps hỗ trợ:
 - Hiển thị bản đồ tương tác.
 - Đánh dấu vị trí (markers) như cửa hàng, nhà ở.
 - Tính toán và vẽ đường đi giữa các điểm.
 - Tích hợp định vị thời gian thực (real-time location)
- Google Maps Platform là bộ công cụ dành cho lập trình viên, bao gồm các API và SDK:
 - Maps SDK for Android/iOS: nhúng bản đồ vào ứng dụng di động.
 - Directions API: tính toán tuyến đường.
 - Geocoding API: chuyển đổi địa chỉ thành tọa độ (và ngược lại).

- Places API: tìm kiếm địa điểm.
- Chi phí: miễn phí đến mức sử dụng nhất định (hiện tại là 200 USD miễn phí mỗi tháng), sau đó tính phí dựa trên số lượng request.
- API Key là một mã định danh duy nhất để:
 - Xác thực ứng dụng của người dùng khi gọi API Google Maps.
 - Theo dõi mức sử dụng và tính phí (nếu vượt ngưỡng miễn phí).
 - Giới hạn quyền truy cập để tăng bảo mật
- Để tích hợp Google Maps, ta sử dụng package **google_maps_flutter**
- Các thành phần chính:
 - **GoogleMap**: Widget hiển thị bản đồ.
 - **Marker**: Đánh dấu vị trí cụ thể trên bản đồ.
 - **Polyline**: Vẽ đường đi giữa các điểm

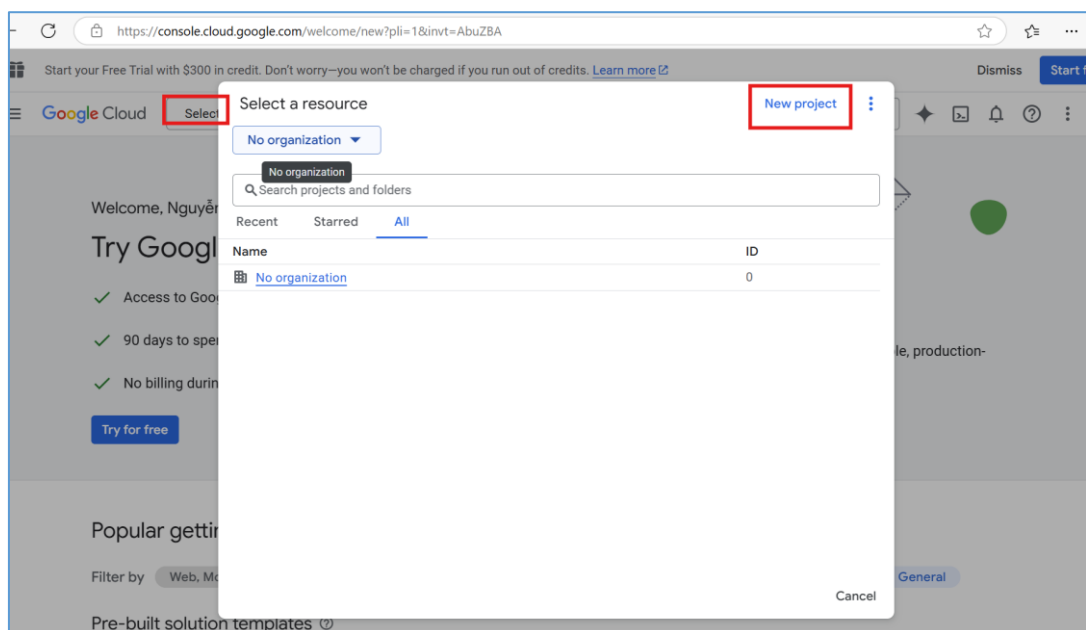
1.2. Hướng dẫn đăng ký API Key của Google Maps

Bước 1: Truy cập Google Cloud Console

1. Mở trình duyệt và truy cập [Google Cloud Console](https://console.cloud.google.com/).
2. Đăng nhập bằng tài khoản Google của bạn (Gmail).

Bước 2: Tạo dự án mới

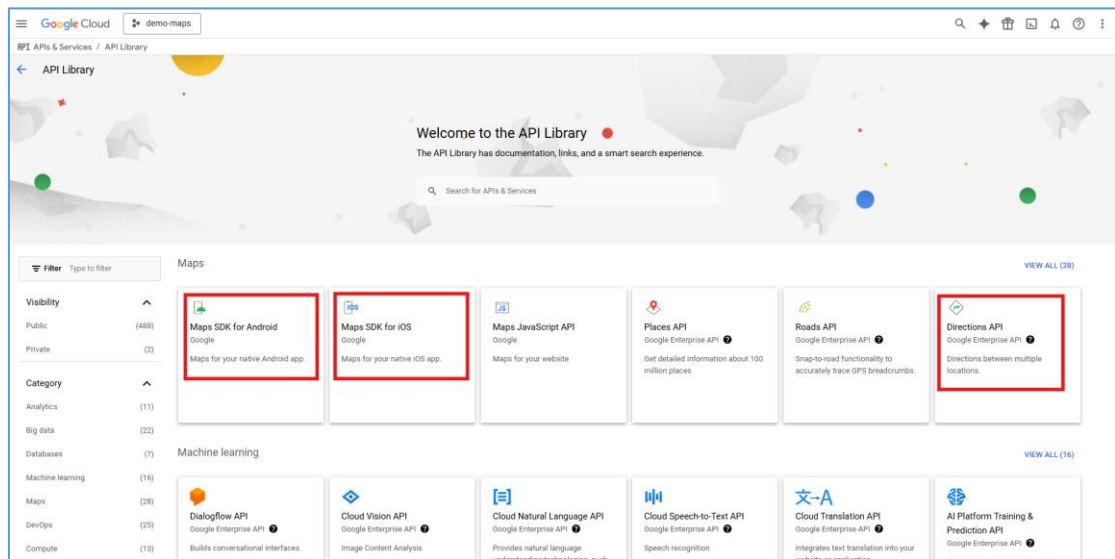
1. Bên cạnh "Google Cloud", chọn Select a project → Xuất hiện cửa sổ, chọn New project.



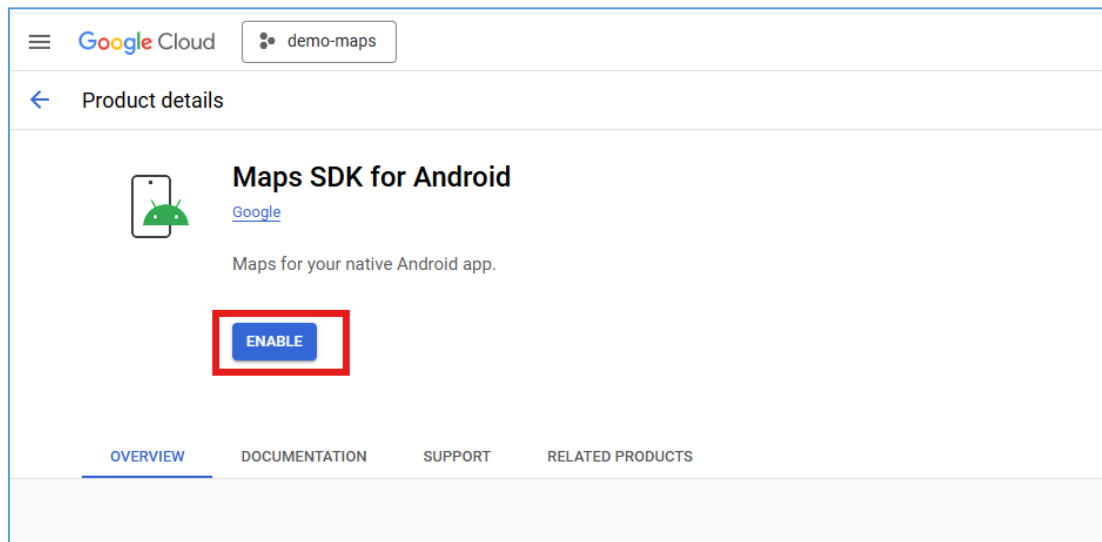
2. Điền thông tin:
 - **Project Name**: Ví dụ: "demo-maps".
 - **Organization** (tùy chọn): Để trống nếu không có tổ chức.
3. Nhấn **Create** (Tạo).

Bước 3: Kích hoạt API Google Maps

1. Vào **APIs & Services > Library** trong menu bên trái.
2. Tìm các API cần thiết:
 - **Maps SDK for Android:** Dùng cho ứng dụng Android.
 - **Maps SDK for iOS:** Dùng cho ứng dụng iOS.
 - **Directions API:** Tìm đường đi giữa các điểm.
3. Nhấp vào từng API và nhấn **Enable** (Kích hoạt) để bật.

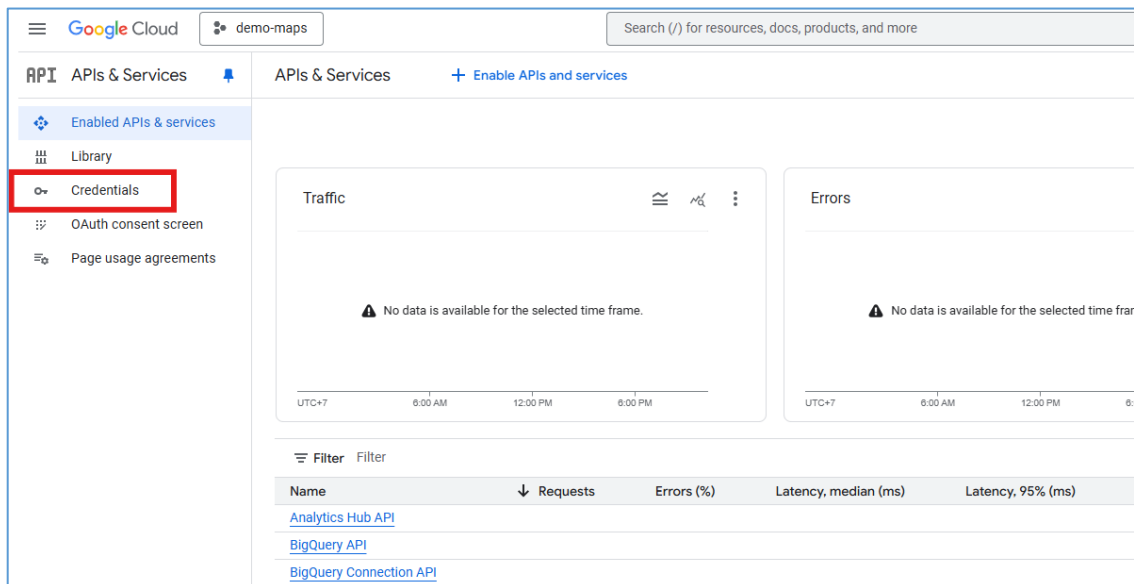


Nếu sau khi click Enable, hệ thống chuyển sang yêu cầu thông tin để thanh toán trong trường hợp người dùng sử dụng dịch vụ vượt ngưỡng miễn phí. Nếu người dùng chỉ sử dụng dưới ngưỡng thì bỏ qua bước này, quay lại APIs & Services của project để thực hiện các bước tiếp theo.

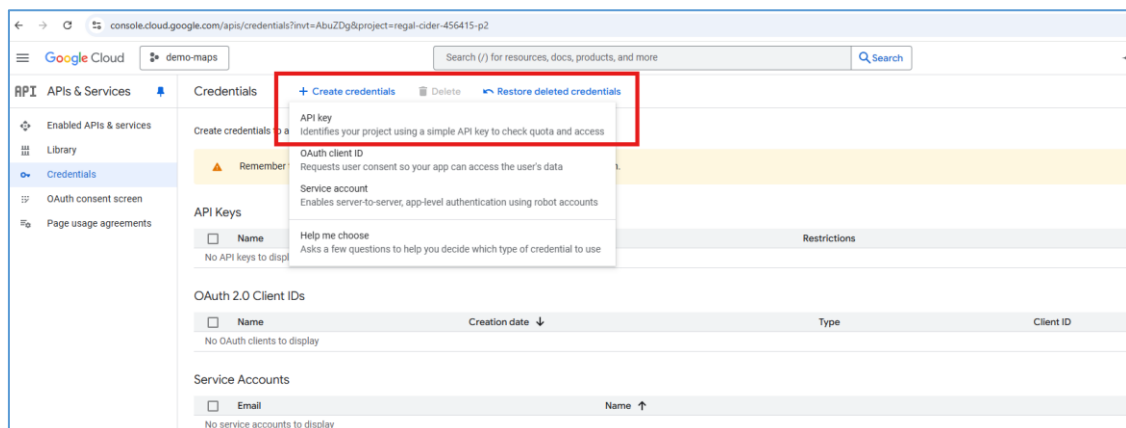


Bước 4: Tạo API Key

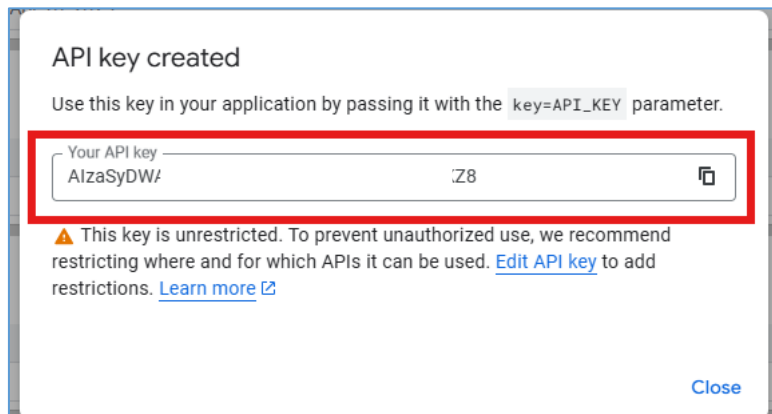
1. Vào **APIs & Services > Credentials** trong menu bên trái.
2. Nhấn nút **+ Create Credentials** (Tạo thông tin xác thực) ở trên cùng.



3. Chọn **API Key** từ menu thả xuống.



4. Một cửa sổ sẽ hiện ra với API Key vừa tạo

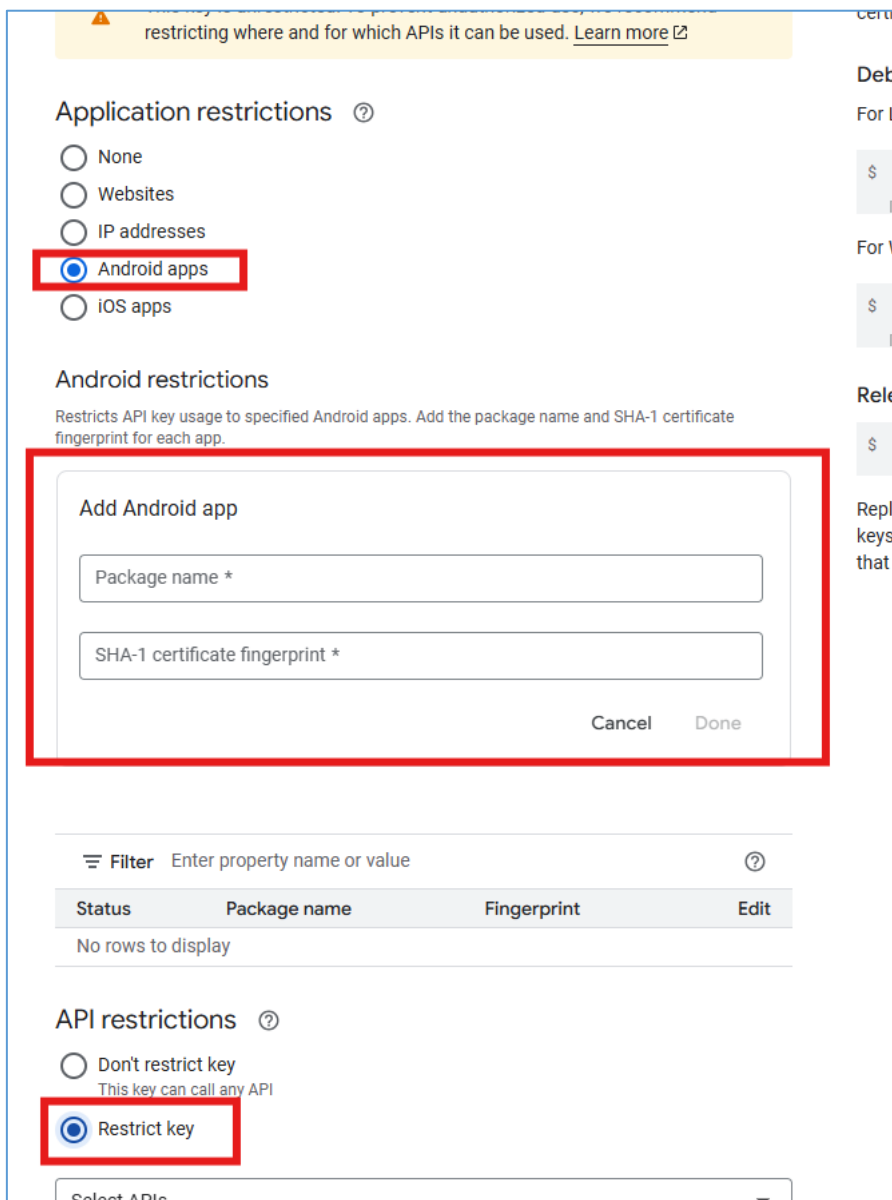


- Sao chép API Key này và lưu lại (vào Notepad hoặc nơi an toàn).

Bước 5: Hạn chế API Key (Tăng bảo mật)

API Key mặc định không bị giới hạn, dễ bị lạm dụng. Để bảo vệ:

1. Trong trang **Credentials**, nhấp vào API Key vừa tạo để chỉnh sửa.



2. Application restrictions (Hạn chế ứng dụng):

- Chọn **Android apps** (hoặc iOS apps nếu cần).

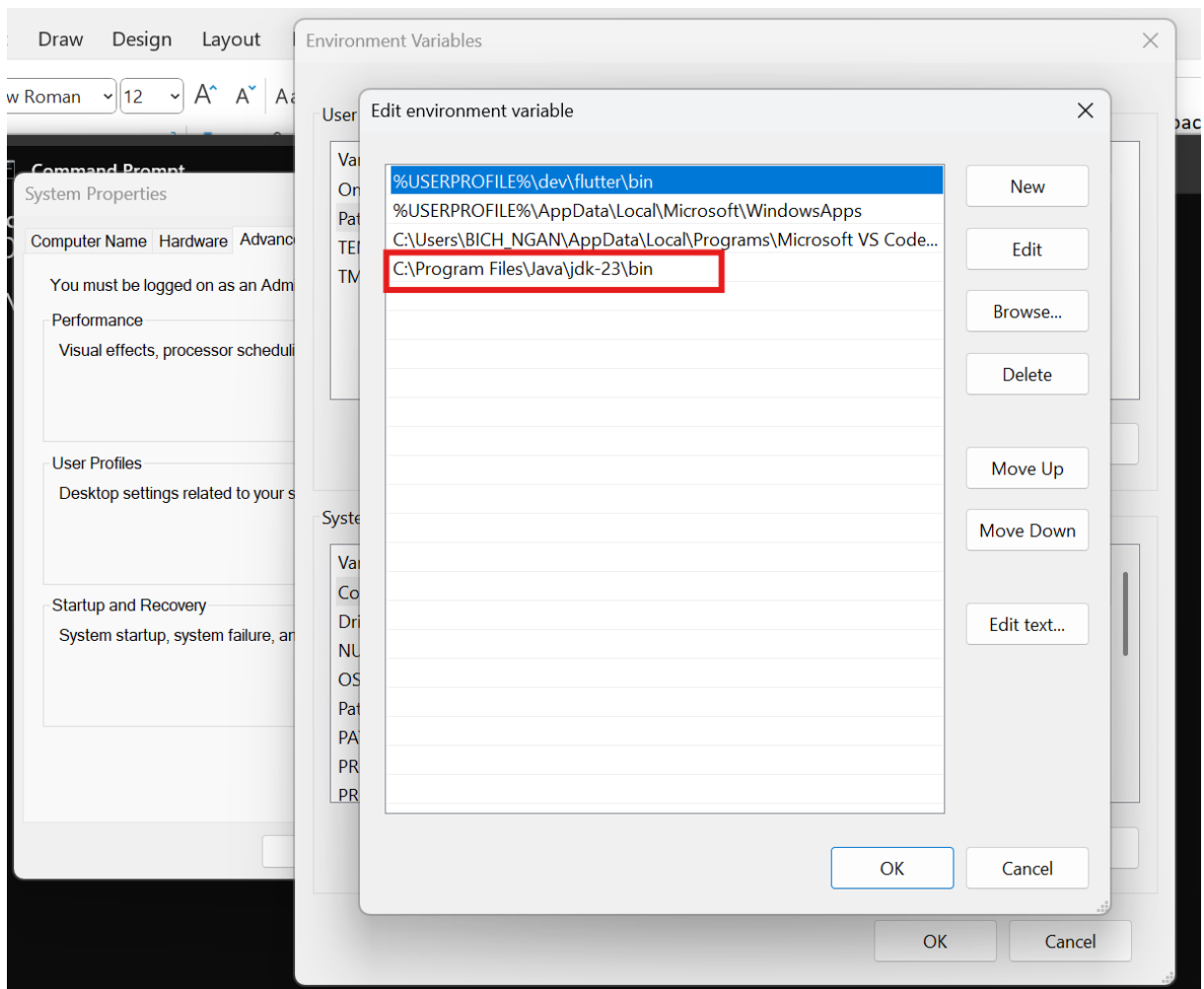
- Nhấn + Add an item và điền:

- **Package Name:** Lấy từ *android/app/build.gradle*



- **SHA-1 Certificate Fingerprint:**

Để lấy SHA-1 cần đảm bảo hệ điều hành đã lưu đường dẫn của Java JDK, như hình dưới:

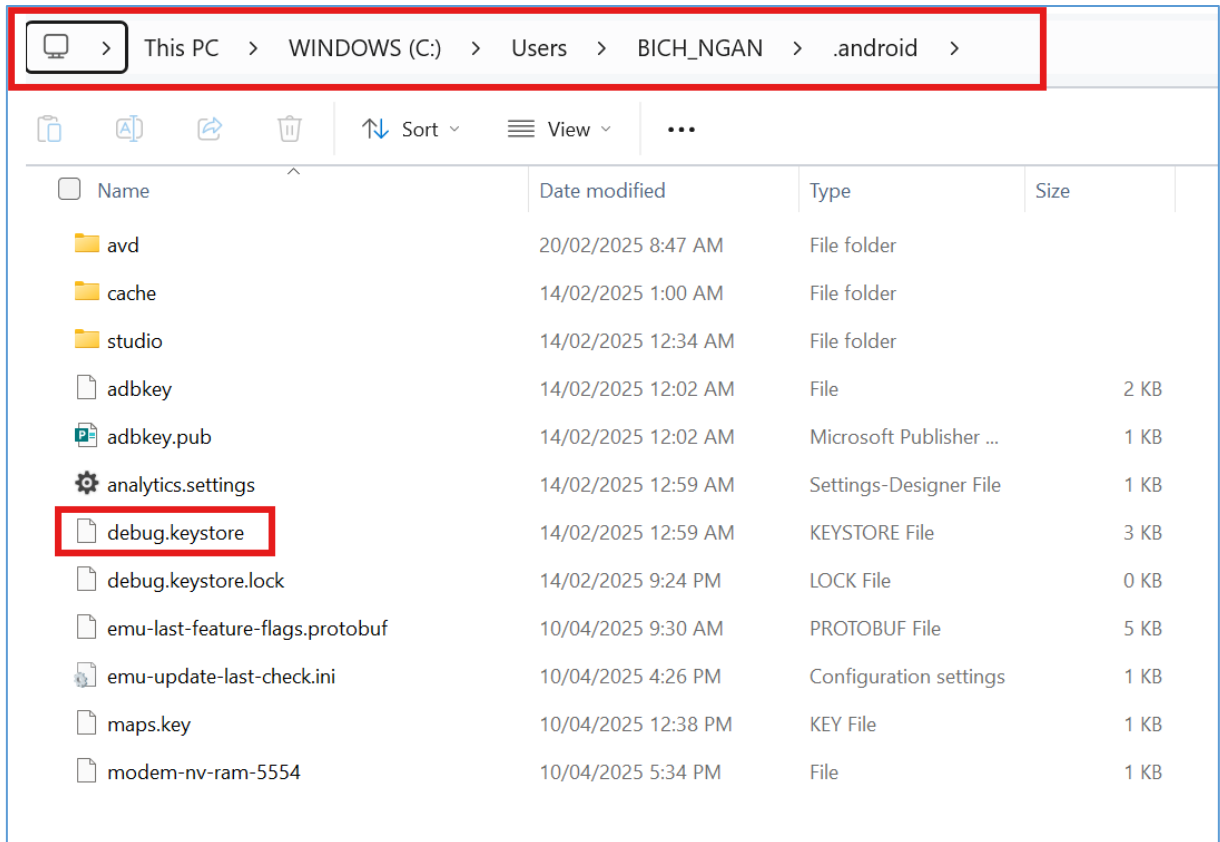


Lấy SHA-1 Certificate Fingerprint bằng lệnh sau (thực thi trong Command Prompt)

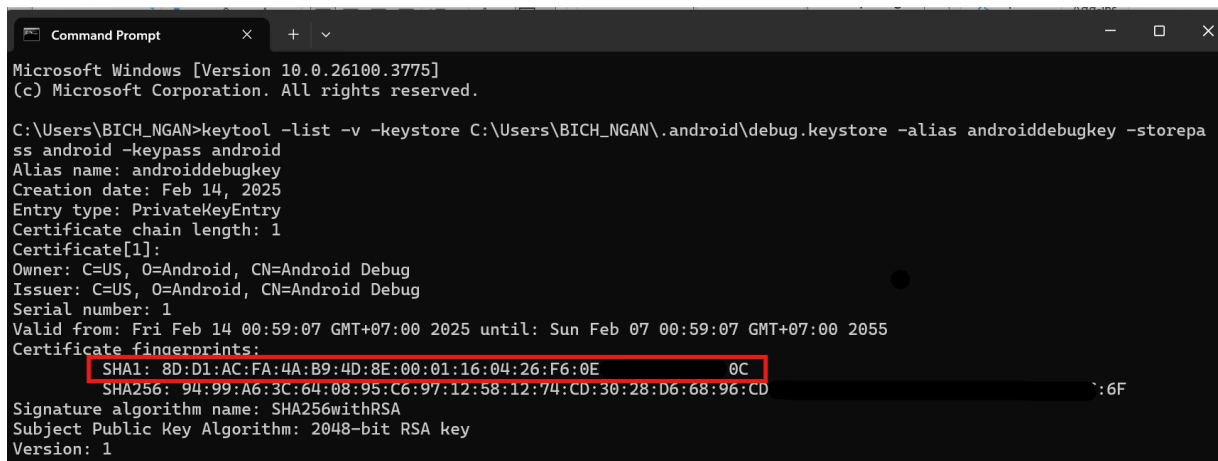
```
keytool -list -v -keystore %USERPROFILE%.android\debug.keystore -alias  
androiddebugkey -storepass android -keypass android
```

UserProfile là đường dẫn đến file debug.keystore trong hệ thống.

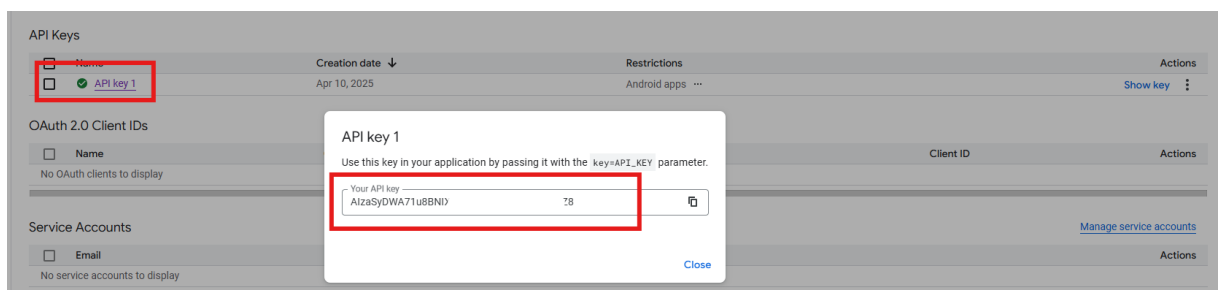
Ví dụ: `keytool -list -v -keystore C:\Users\<Tên người dùng>\.android\debug.keystore -alias androiddebugkey -storepass android -keypass android`



Kết quả SHA-1

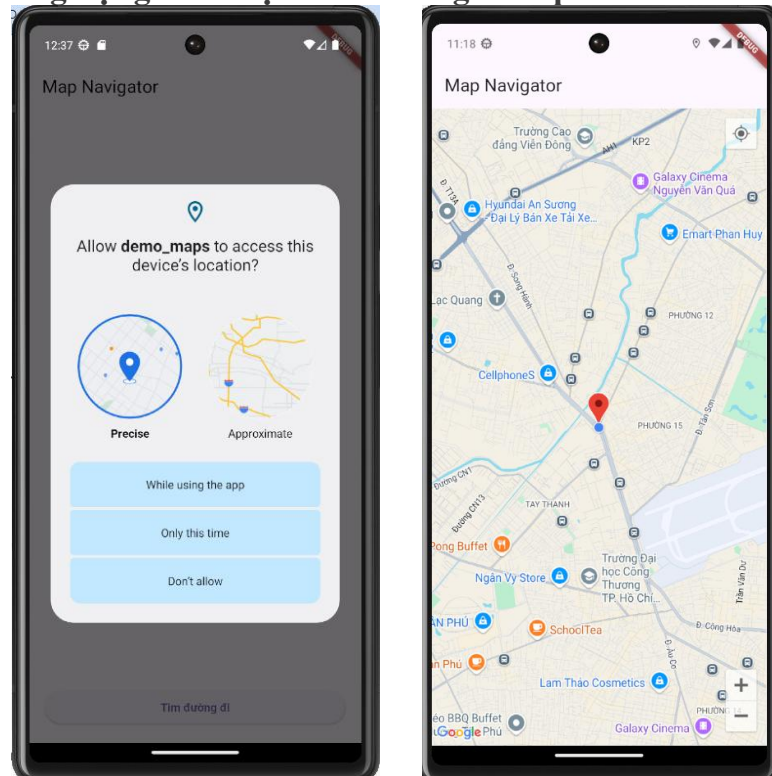


Kết quả API Key sau khi kích hoạt.



2. Bài tập hướng dẫn

Bài tập 1. Tạo ứng dụng hiển thị bản đồ Google Maps và đánh dấu vị trí hiện tại.

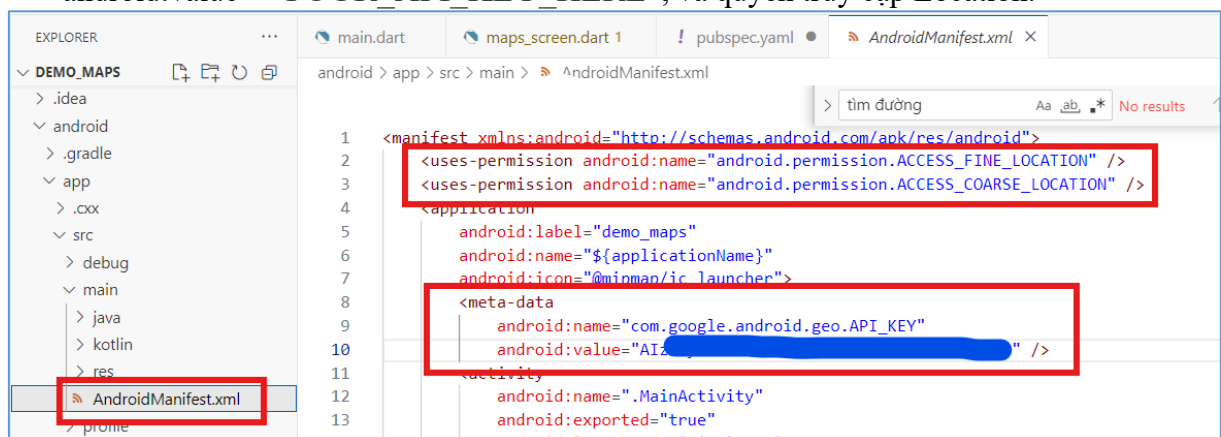


Hướng dẫn

- Bước 1: lấy API key từ Google.
- Bước 2: Thêm các package cần thiết:

```
dependencies:  
  flutter:  
    sdk: flutter  
  google_maps_flutter: ^2.7.0    # Hiển thị Google Maps  
  geolocator: ^12.0.0           # Lấy vị trí hiện tại  
  http: ^1.2.1                  # Gửi yêu cầu HTTP
```

- Bước 3: Cấu hình API key cho Android:
 - ❖ Đối với Android, mở file **android/app/src/main/AndroidManifest.xml** và thêm API Key tại **android:value= "YOUR_API_KEY_HERE"**, và quyền truy cập Location.



- ❖ Đối với iOS, mở file **ios/Runner/AppDelegate.swift** và thêm

```
import UIKit  
import GoogleMaps
```

```

@UIApplicationMain
@objc class AppDelegate: FlutterAppDelegate {
  override func application(
    _ application: UIApplication,
    didFinishLaunchingWithOptions launchOptions:
[UIApplication.LaunchOptionsKey: Any]?
  ) -> Bool {
    GMSServices.provideAPIKey("YOUR_API_KEY_HERE") // Thay bằng API Key
    return super.application(application, didFinishLaunchingWithOptions:
launchOptions)
  }
}

```

Mở file **ios/Runner/Info.plist** và thêm quyền:

```

<key>NSLocationWhenInUseUsageDescription</key>
<string>Ứng dụng cần vị trí để hiển thị bản đồ.</string>

```

- Bước 4. Cài đặt source code:

- Mở file **lib/main.dart**

```

import 'package:flutter/material.dart';
import 'maps_screen.dart';

void main() {
  runApp(MapNavigatorApp());
}

class MapNavigatorApp extends StatelessWidget {
  const MapNavigatorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(title: 'Map Navigator', home: MapScreen());
  }
}

```

- Tạo file **lib/maps_screen.dart**

```

import 'dart:async';
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'package:geolocator/geolocator.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class MapScreen extends StatefulWidget {
  const MapScreen({super.key});

  @override
  _MapScreenState createState() => _MapScreenState();
}

class _MapScreenState extends State<MapScreen> {

```

```

Completer<GoogleMapController> _controller = Completer();
Set<Marker> _markers = {};
Set<Polyline> _polylines = {};
LatLng? _currentPosition;
LatLng _destination = LatLng(10.7769, 106.7009); // Ví dụ: TP.HCM

static final CameraPosition _initialPosition = CameraPosition(
  target: LatLng(10.7769, 106.7009), // Vị trí mặc định: TP.HCM
  zoom: 14,
);

@override
void initState() {
  super.initState();
  _getCurrentLocation();
}

// Lấy vị trí hiện tại của người dùng
Future<void> _getCurrentLocation() async {
  bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
  if (!serviceEnabled) {
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text('Vui lòng bật dịch vụ vị trí!')));
    return;
  }

  LocationPermission permission = await Geolocator.checkPermission();
  if (permission == LocationPermission.denied) {
    permission = await Geolocator.requestPermission();
    if (permission == LocationPermission.denied) return;
  }

  Position position = await Geolocator.getCurrentPosition(
    desiredAccuracy: LocationAccuracy.high,
  );
  setState(() {
    _currentPosition = LatLng(position.latitude, position.longitude);
    _addMarker(_currentPosition!, "Vị trí của bạn");
    _moveCamera(_currentPosition!);
  });
}

// Thêm marker
void _addMarker(LatLng position, String markerId) {
  setState(() {
    _markers.add(
      Marker(
        markerId: MarkerId(markerId),
        position: position,
        infoWindow: InfoWindow(title: markerId),
      ),
    ),
  );
}

```

```

    );
  });
}

// Di chuyển camera đến vị trí
Future<void> _moveCamera(LatLng position) async {
  final GoogleMapController controller = await _controller.future;
  controller.animateCamera(CameraUpdate.newLatLng(position));
}

// Tìm đường đi
Future<void> _getDirections() async {
  if (_currentPosition == null) return;

  String apiKey =
    "AIzaSyAdST0k5BTg3dQbKyguU1N-A_wqA7YoiD8"; // Thay bằng API Key
  String url =
    "https://maps.googleapis.com/maps/api/directions/json?origin=${_currentPosition!.latitude},${_currentPosition!.longitude}&destination=${_destination.latitude},${_destination.longitude}&key=$apiKey";

  final response = await http.get(Uri.parse(url));
  if (response.statusCode == 200) {
    Map<String, dynamic> data = jsonDecode(response.body);

    if (data['routes'] != null && data['routes'].isNotEmpty) {
      String polylinePoints =
        data['routes'][0]['overview_polyline']['points'];
      List<LatLng> points = _decodePolyline(polylinePoints);

      setState(() {
        _polylines.add(
          Polyline(
            polylineId: PolylineId('route'),
            points: points,
            color: Colors.blue,
            width: 5,
          ),
        );
        _addMarker(_destination, "Đích đến");
      });
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Không tìm thấy tuyến đường nào!')),
      );
    }
  } else {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Lỗi khi gọi API: ${response.statusCode}')),
    );
  }
}
}

```

```

// Giải mã polyline từ Directions API
List<LatLng> _decodePolyline(String encoded) {
  List<LatLng> points = [];
  int index = 0, len = encoded.length;
  int lat = 0, lng = 0;

  while (index < len) {
    int b, shift = 0, result = 0;
    do {
      b = encoded.codeUnitAt(index++) - 63;
      result |= (b & 0x1f) << shift;
      shift += 5;
    } while (b >= 0x20);
    int dlat = ((result & 1) != 0 ? ~(result >> 1) : (result >> 1));
    lat += dlat;

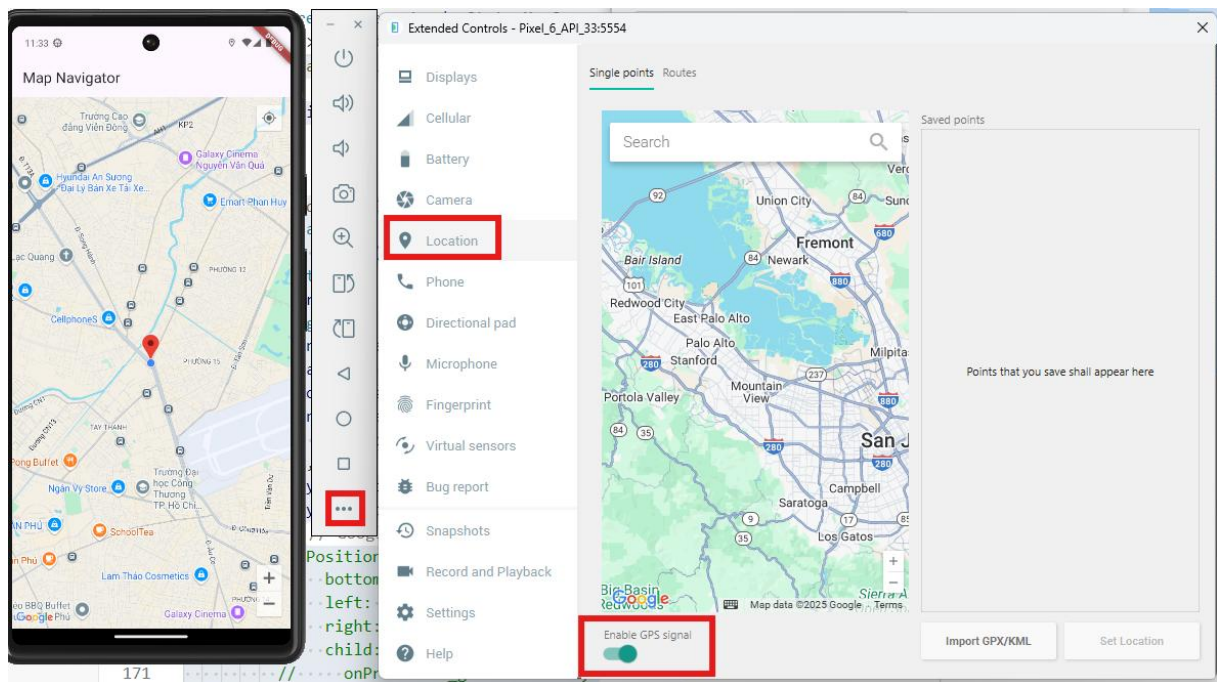
    shift = 0;
    result = 0;
    do {
      b = encoded.codeUnitAt(index++) - 63;
      result |= (b & 0x1f) << shift;
      shift += 5;
    } while (b >= 0x20);
    int dlng = ((result & 1) != 0 ? ~(result >> 1) : (result >> 1));
    lng += dlng;

    points.add(LatLng(lat / 1E5, lng / 1E5));
  }
  return points;
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text('Map Navigator')),
    body: Stack(
      children: [
        GoogleMap(
          initialCameraPosition: _initialPosition,
          markers: _markers,
          polylines: _polylines,
          onMapCreated: (GoogleMapController controller) {
            _controller.complete(controller);
          },
          myLocationEnabled: true,
          myLocationButtonEnabled: true,
        ),
      ],
    ),
  );
}

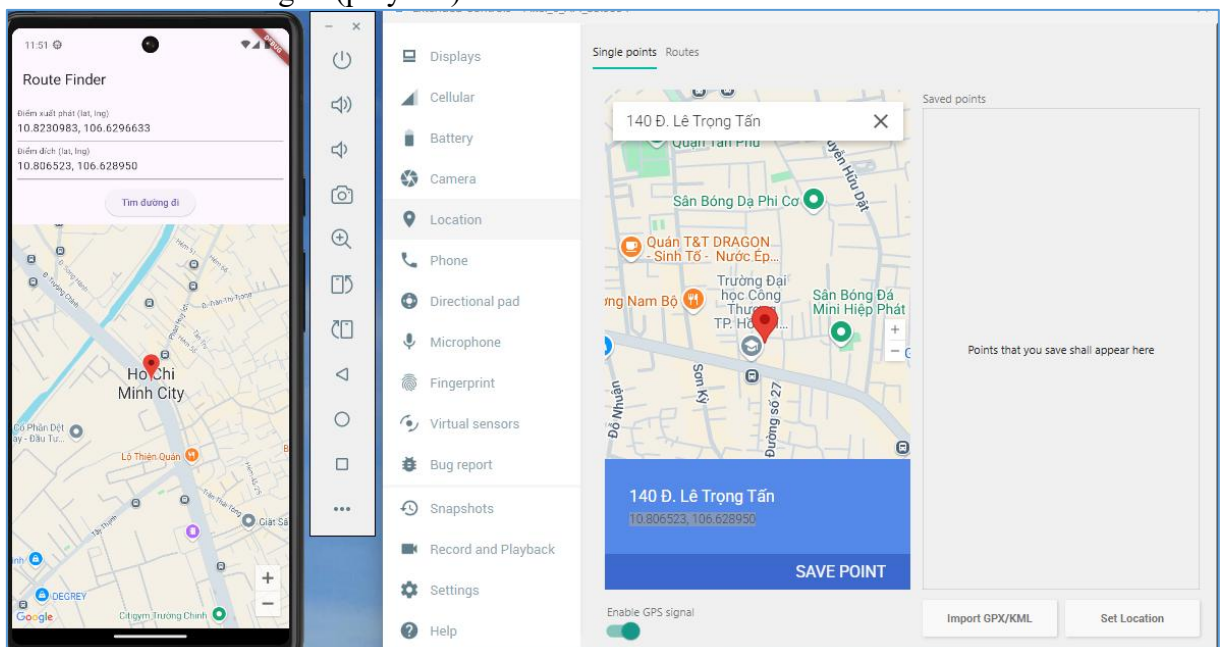
```

Để thực thi chương trình cần bật GPS cho thiết bị trước. Cách bật GPS cho máy ảo như hình dưới:



Bài tập 2. Bổ sung vào bài tập 1 các chức năng sau:

- Nhập hai điểm (xuất phát và đích đến) để tìm đường đi.
- Đánh dấu marker tại hai điểm.
- Vẽ đường đi (polyline) trên bản đồ



Gợi ý: Thực hiện các bước lấy API key và package như bài 1.

Source code

File main:

```
import 'package:flutter/material.dart';
import 'router_finder_screen.dart';
```

```
void main() {
```

Trang 13

```

    // runApp(MapNavigatorApp());
    runApp(RouteFinderApp());
  }

class RouteFinderApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(title: 'Route Finder', home: RouteFinderScreen());
  }
}

```

File 'router_finder_screen.dart'

```

import 'dart:async';
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'package:geolocator/geolocator.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class RouteFinderScreen extends StatefulWidget {
  @override
  _RouteFinderScreenState createState() => _RouteFinderScreenState();
}

class _RouteFinderScreenState extends State<RouteFinderScreen> {
  Completer<GoogleMapController> _controller = Completer();
  Set<Marker> _markers = {};
  Set<Polyline> _polylines = {};
  TextEditingController _startController = TextEditingController();
  TextEditingController _endController = TextEditingController();
  LatLng? _startLatLng;
  LatLng? _endLatLng;

  static final CameraPosition _initialPosition = CameraPosition(
    target: LatLng(10.7769, 106.7009), // TP.HCM mặc định
    zoom: 12,
  );

  @override
  void initState() {
    super.initState();
    _getCurrentLocation();
  }

  // Lấy vị trí hiện tại
  Future<void> _getCurrentLocation() async {
    LocationPermission permission = await Geolocator.requestPermission();
    if (permission == LocationPermission.denied) return;

    Position position = await Geolocator.getCurrentPosition();
    setState(() {

```

```

        _startLatLng = LatLng(position.latitude, position.longitude);
        _addMarker(_startLatLng!, "Xuất phát");
        _moveCamera(_startLatLng!);
        _startController.text = "${position.latitude}, ${position.longitude}";
    });
}

// Thêm marker
void _addMarker(LatLng position, String markerId) {
    setState(() {
        _markers.add(
            Marker(
                markerId: MarkerId(markerId),
                position: position,
                infoWindow: InfoWindow(title: markerId),
            ),
        );
    });
}

// Di chuyển camera
Future<void> _moveCamera(LatLng position) async {
    final GoogleMapController controller = await _controller.future;
    controller.animateCamera(CameraUpdate.newLatLng(position));
}

// Tìm đường đi
Future<void> _findRoute() async {
    if (_startController.text.isEmpty || _endController.text.isEmpty) {
        ScaffoldMessenger.of(
            context,
        ).showSnackBar(SnackBar(content: Text('Vui lòng nhập cả hai điểm!')));
        return;
    }

    List<String> start = _startController.text.split(',');
    List<String> end = _endController.text.split(',');
    _startLatLng = LatLng(double.parse(start[0]), double.parse(start[1]));
    _endLatLng = LatLng(double.parse(end[0]), double.parse(end[1]));

    String apiKey =
        "YOUR API KEY"; // Thay bằng API Key
    String url =
        "https://maps.googleapis.com/maps/api/directions/json?origin=${_startL
atLng!.latitude},${_startLatLng!.longitude}&destination=${_endLatLng!.latitude
},${_endLatLng!.longitude}&key=$apiKey";

    final response = await http.get(Uri.parse(url));
    if (response.statusCode == 200) {
        Map<String, dynamic> data = jsonDecode(response.body);

        if (data['routes'] != null && data['routes'].isNotEmpty) {

```

```

String polylinePoints =
    data['routes'][0]['overview_polyline']['points'];
List<LatLng> points = _decodePolyline(polylinePoints);

setState(() {
    _markers.clear();
    _addMarker(_startLatLng!, "Xuất phát");
    _addMarker(_endLatLng!, "Đích đến");
    _polylines.clear();
    _polylines.add(
        Polyline(
            polylineId: PolylineId('route'),
            points: points,
            color: Colors.blue,
            width: 5,
        ),
    );
    _moveCamera(_startLatLng!);
});
} else {
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Không tìm thấy tuyến đường nào!')),
    );
}
} else {
    ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Lỗi khi gọi API: ${response.statusCode}')),
    );
}
}

// Giải mã polyline
List<LatLng> _decodePolyline(String encoded) {
    List<LatLng> points = [];
    int index = 0, len = encoded.length;
    int lat = 0, lng = 0;

    while (index < len) {
        int b, shift = 0, result = 0;
        do {
            b = encoded.codeUnitAt(index++) - 63;
            result |= (b & 0x1f) << shift;
            shift += 5;
        } while (b >= 0x20);
        int dlat = ((result & 1) != 0 ? ~(result >> 1) : (result >> 1));
        lat += dlat;

        shift = 0;
        result = 0;
        do {
            b = encoded.codeUnitAt(index++) - 63;
            result |= (b & 0x1f) << shift;

```

```

        shift += 5;
    } while (b >= 0x20);
    int dlng = ((result & 1) != 0 ? ~(result >> 1) : (result >> 1));
    lng += dlng;

    points.add(LatLng(lat / 1E5, lng / 1E5));
}
return points;
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: Text('Route Finder')),
        body: Column(
            children: [
                Padding(
                    padding: EdgeInsets.all(8.0),
                    child: Column(
                        children: [
                            TextField(
                                controller: _startController,
                                decoration: InputDecoration(
                                    labelText: 'Điểm xuất phát (lat, lng)',
                                ),
                            ),
                            TextField(
                                controller: _endController,
                                decoration: InputDecoration(
                                    labelText: 'Điểm đích (lat, lng)',
                                ),
                            ),
                            SizedBox(height: 10),
                            ElevatedButton(
                                onPressed: _findRoute,
                                child: Text('Tìm đường đi'),
                            ),
                        ],
                    ),
                ),
                Expanded(
                    child: GoogleMap(
                        initialCameraPosition: _initialPosition,
                        markers: _markers,
                        polylines: _polylines,
                        onMapCreated: (GoogleMapController controller) {
                            _controller.complete(controller);
                        },
                        myLocationEnabled: true,
                    ),
                ),
            ],
        ),
    );
}

```

```
    },  
    );  
}  
}
```

3. Bài tập trên lớp

Bài tập 4. Bổ sung chức năng cho bài tập 3

- Thêm nút lấy vị trí hiện tại làm điểm đích.
- Hiển thị khoảng cách và thời gian di chuyển từ Directions API.
- Cho phép chọn điểm trên bản đồ bằng cách click chuột.
- Cho tìm kiếm địa điểm như khách sạn, quán ăn, bệnh viện, trường học,...

Gợi ý:

- Dùng các thư viện: google_maps_flutter, geolocator, Google Places API

4. Bài tập về nhà

Bổ sung thêm các chức năng :

- Tìm kiếm địa chỉ bằng Geocoding API, nghĩa là thay vì nhập tọa độ, cho phép người dùng nhập địa chỉ (ví dụ: "227 Nguyễn Văn Cừ, TP.HCM") và chuyển thành tọa độ.
- Cho phép người dùng chọn loại phương tiện (xe hơi, đi bộ, xe máy) khi tìm đường đi. Gợi ý : dùng dữ liệu của Directions API.
- Vẽ đường đi trên bản đồ.
- Tìm tuyến đường tối ưu cho đi bộ, xe máy, ô tô.
- Thêm tính năng lưu tuyến đường đã tìm vào danh sách yêu thích và hiển thị lại khi cần. Gợi ý : dùng SQLite để lưu dữ liệu tuyến đường yêu thích.

-- HẾT--